TP4 Android

Gestion complète des événements
(Application SalleGM)

BTS Développement Mobile

Objectifs du TP

Ce TP a pour objectif de mettre en pratique **l'ensemble des notions liées à la gestion des événements Android** vues en cours.

À la fin de ce TP, l'étudiant sera capable de :

- Créer une application Android multi-écrans
- Gérer un **login** avec validation
- Utiliser différents **listeners** (clic, sélection, saisie, état)
- Utiliser un **popup (AlertDialog)**
- Naviguer entre les interfaces via des **Intents**
- Comprendre le lien entre **événement** et **action**

1 Contexte de l'application

Vous développez une application appelée **SalleGM** destinée à la **gestion des salles** d'un établissement.

L'application permet :

- l'authentification d'un gestionnaire,
- l'ajout et la réservation d'une salle,
- l'activation ou non d'équipements,
- la confirmation des actions via des popups.

2 Architecture de l'application

L'application contient **4 interfaces** :

1. **LoginActivity**
2. **DashboardActivity**
3. **SalleActivity**
4. **ConfirmationActivity**

La navigation entre les interfaces se fait via des **Intents explicites** déclenchés par des événements.

Liberté de conception graphique Vous êtes libres de concevoir les interfaces de votre application comme vous le souhaitez, tant au niveau de la disposition des éléments, des couleurs que du thème général de l'application.

Vous pouvez choisir le domaine qui vous convient (gestion de salles, gestion d'événements, gestion de club, etc.) et enrichir vos interfaces à l'aide d'icônes et d'images libres de droits.

À cet effet, vous pouvez utiliser les plateformes suivantes :

— **Flaticon** (https ://www.flaticon.com) pour les icônes

— **Unsplash** (https ://unsplash.com), **Pexels** (https ://www.pexels.com) ou **Pixabay** (https ://-pixabay.com) pour les images.

Le critère principal d'évaluation de ce TP reste la bonne gestion des événements (listeners, callbacks, Intents, popup), et non l'aspect graphique.

3 Interface 1 : Login

Composants

— EditText : Login

— EditText : Mot de passe

— Button : Se connecter

Gestion des événements

— **OnClickListener** sur le bouton

— Vérification des champs

— Toast si champs vides

— Intent vers Dashboard si valide

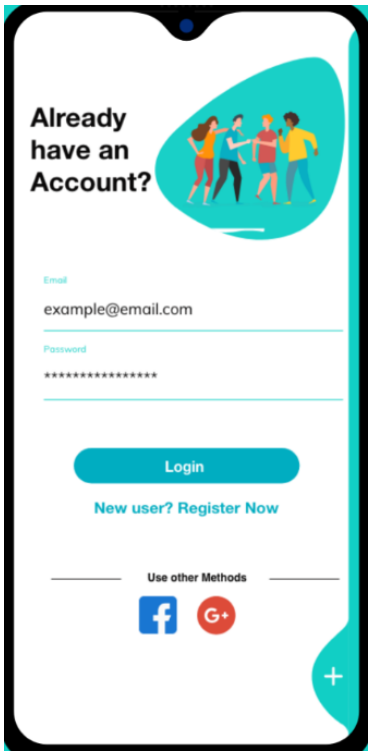

FIGURE 1 – Exemple d'interface de Login

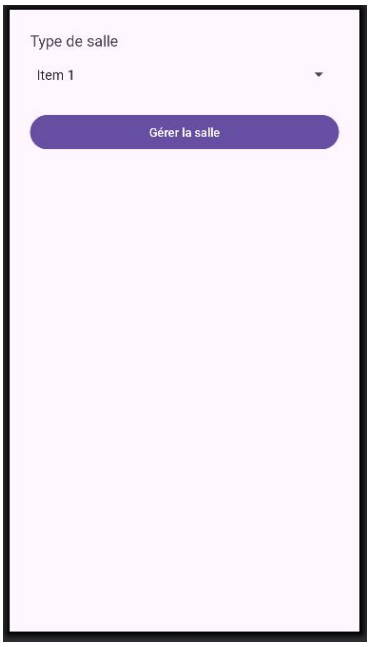

FIGURE 2 – Exemple d'interface de Dashboard

4 Interface 2 : Dashboard

Composants

- TextView : message de bienvenue
- Spinner : type de salle (TD, TP, Amphi)
- Button : Gérer la salle

Gestion des événements

- **OnItemSelectedListener** sur le Spinner
- **OnClickListener** sur le bouton
- Passage du type de salle via Intent

```
public class DashboardActivity extends AppCompatActivity {


    Spinner spinnerSalle;
    Button btnManage;
    String typeSalle = "";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_dashboard);

        spinnerSalle = findViewById(R.id.spinnerSalle);
        btnManage = findViewById(R.id.btnManage);

        //        Donn es du Spinner
        String[] salles = {"TD", "TP", "Amphi"};

        //        Adapter
```

```java
        ArrayAdapter<String> adapter = new ArrayAdapter<>(
                this,
                android.R.layout.simple_spinner_item,
                salles
        );
        adapter.setDropDownViewResource(
                android.R.layout.simple_spinner_dropdown_item
        );

        //        Lier Spinner     Adapter
        spinnerSalle.setAdapter(adapter);

        //        Listener
        spinnerSalle.setOnItemSelectedListener(
                new AdapterView.OnItemSelectedListener() {
                    @Override
                    public void onItemSelected(
                            AdapterView<?> parent,
                            View view,
                            int position,
                            long id) {

                        typeSalle = parent
                                .getItemAtPosition(position)
                                .toString();
                    }

                    @Override
                    public void onNothingSelected(
                            AdapterView<?> parent) {
                    }
                });

        btnManage.setOnClickListener(v -> {
            Intent i = new Intent(
                    DashboardActivity.this,
                    SalleActivity.class
            );
            i.putExtra("typeSalle", typeSalle);
            startActivity(i);
        });
    }
}
```

5 Notion clé : Adapter (Spinner)

Dans Android, un composant graphique comme le **Spinner** ne peut pas afficher directement des données. Il a besoin d'un intermédiaire appelé **Adapter**.

Rôle de l'Adapter

L'Adapter joue le rôle de **pont** entre :

— les **données** (tableau, liste, base de données),

— et le **composant graphique** (Spinner, ListView, etc.).

Sans Adapter :

— le Spinner s'affiche,
— mais il reste **vide**,
— aucun élément ne peut être sélectionné.

Fonctionnement du Spinner avec Adapter

Le fonctionnement se fait en **3 étapes obligatoires** :

1. Définir les données à afficher (exemple : types de salles)
2. Créer un `ArrayAdapter`
3. Associer l'Adapter au Spinner avec `setAdapter()`

Exemple de code

```
String[] salles = {"TD", "TP", "Amphi"};

ArrayAdapter<String> adapter = new ArrayAdapter<>(
        this,
        android.R.layout.simple_spinner_item,
        salles
);

adapter.setDropDownViewResource(
        android.R.layout.simple_spinner_dropdown_item
);

spinnerSalle.setAdapter(adapter);
```

Lien avec les événements

Une fois l'Adapter associé :

— le Spinner affiche les éléments,
— l'utilisateur peut sélectionner une valeur,
— l'événement `OnItemSelectedListener` est déclenché.

Conclusion

Un Spinner ne peut jamais fonctionner sans Adapter. L'Adapter est indispensable pour afficher les données et activer la gestion des événements.

6 Interface 3 : Gestion de salle

Composants

— EditText : Nom de la salle
— Switch : Vidéoprojecteur
— CheckBox : Climatisation
— CheckBox : Accès Wi-Fi
— Button : Valider réservation

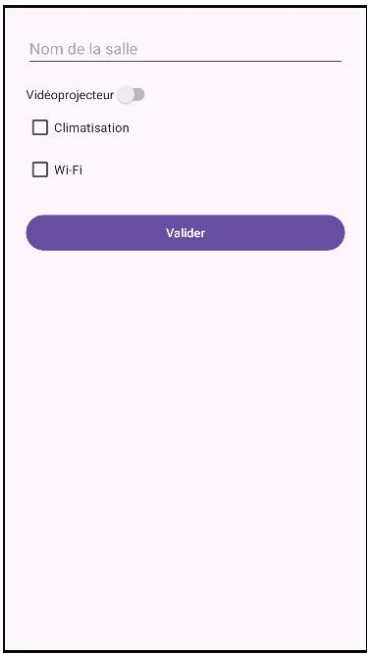

FIGURE 3 – Exemple d'interface de Gestion de salle

Gestion des événements

- **TextWatcher** sur le nom de la salle
- **OnCheckedChangeListener** sur Switch et CheckBox
- **OnClickListener** sur Valider

```
public class SalleActivity extends AppCompatActivity {



    EditText etNom;
    Switch switchVideo;
    CheckBox cbClim, cbWifi;
    Button btnValider;

    boolean video = false, clim = false, wifi = false;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_salle);

        etNom = findViewById(R.id.etNomSalle);
        switchVideo = findViewById(R.id.switchVideo);
        cbClim = findViewById(R.id.cbClim);
        cbWifi = findViewById(R.id.cbWifi);
        btnValider = findViewById(R.id.btnValider);

        etNom.addTextChangedListener(new TextWatcher() {
            @Override
            public void onTextChanged(
                    CharSequence s, int start, int before, int count) {
            }

```

```
            @Override public void beforeTextChanged(...) {}
            @Override public void afterTextChanged(...) {}
        });

        switchVideo.setOnCheckedChangeListener(
                (buttonView, isChecked) -> video = isChecked);

        cbClim.setOnCheckedChangeListener(
                (buttonView, isChecked) -> clim = isChecked);

        cbWifi.setOnCheckedChangeListener(
                (buttonView, isChecked) -> wifi = isChecked);

        btnValider.setOnClickListener(v -> showPopup());
    }

    private void showPopup() {
        AlertDialog.Builder builder =
                new AlertDialog.Builder(this);

        builder.setTitle("Confirmation");
        builder.setMessage("Confirmer la r servation ?");

        builder.setPositiveButton("Oui", (d, w) -> {
            Intent i = new Intent(
                    SalleActivity.this,
                    ConfirmationActivity.class
            );
            i.putExtra("video", video);
            i.putExtra("clim", clim);
            i.putExtra("wifi", wifi);
            startActivity(i);
        });

        builder.setNegativeButton("Annuler", null);
        builder.show();
    }
}
```

Événement de saisie : TextWatcher

Le `TextWatcher` permet de détecter les **modifications du texte** saisies par l'utilisateur dans un `EditText`.

À chaque changement du texte, Android déclenche automatiquement un événement. Le développeur peut alors réagir à la saisie clavier en temps réel.

Méthodes principales

- `beforeTextChanged()` : appelée avant la modification du texte. Exemple d'utilisation : sauvegarder l'ancien texte, bloquer une modification, préparer une validation.
- `onTextChanged()` : appelée pendant la saisie de l'utilisateur. Exemples concrets : afficher le texte en temps réel, vérifier la longueur, afficher un message si le champ est vide.
- `afterTextChanged()` : appelée après la modification du texte. Exemples utilisation :valider le champ, activer/désactiver un bouton, vérifier un format (ex : nom non vide).

Ces trois méthodes sont obligatoires car `TextWatcher` est une interface Java. Le `TextWatcher` illustre la programmation événementielle : l'application réagit automatiquement aux actions de l'utilisateur sans clic sur un bouton.

```
etNom.addTextChangedListener(new TextWatcher() {
    @Override
    public void beforeTextChanged(
            CharSequence s, int start, int count, int after) {
    }

    @Override
    public void onTextChanged(
            CharSequence s, int start, int before, int count) {
    }

    @Override
    public void afterTextChanged(Editable s) {
        btnValider.setEnabled(s.length() > 0);
    }
});
```

Utilité du TextWatcher Le `TextWatcher` permet de détecter les modifications du texte saisies par l'utilisateur dans un `EditText`. Dans cet exemple, il est utilisé pour activer le bouton de validation uniquement lorsque le champ *Nom de la salle* n'est pas vide. Ainsi, l'application réagit en temps réel à la saisie clavier, sans attendre un clic, ce qui illustre concrètement la programmation événementielle.

7 Popup de confirmation

Avant de confirmer la réservation, un **popup** doit s'afficher.

```
AlertDialog.Builder builder =
    new AlertDialog.Builder(this);

builder.setTitle("Confirmation");
builder.setMessage("Confirmer la r servation ?");

builder.setPositiveButton("Oui", (d, w) -> {
    // action confirm e
});

builder.setNegativeButton("Annuler", (d, w) -> {
    d.dismiss();
});

builder.show();
```

Ce popup est déclenché par un **événement de clic**.

8 Interface 4 : Confirmation

Cette interface affiche :

— Les informations de la salle

— Les équipements sélectionnés

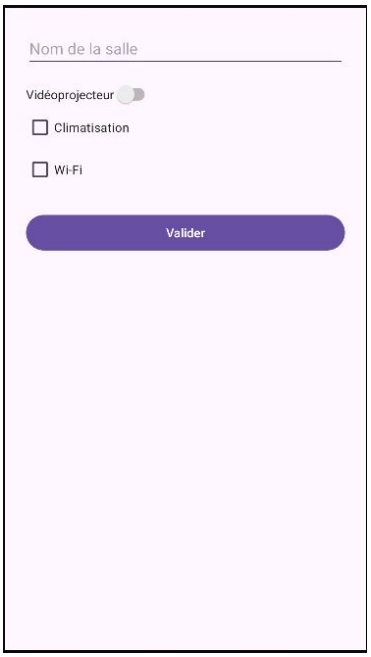

FIGURE 4 – Exemple d'interface de Gestion de salle

Les données sont récupérées via :

- `getIntent().getStringExtra()`
- `getIntent().getBooleanExtra()`

```
public class ConfirmationActivity extends AppCompatActivity {

    TextView tvResume;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_confirmation);

        tvResume = findViewById(R.id.tvResume);

        boolean video = getIntent().getBooleanExtra("video", false);
        boolean clim = getIntent().getBooleanExtra("clim", false);
        boolean wifi = getIntent().getBooleanExtra("wifi", false);

        tvResume.setText(
                "Vid oprojecteur : " + video +
                "\nClimatisation : " + clim +
                "\nWiFi : " + wifi
        );
    }
}
```

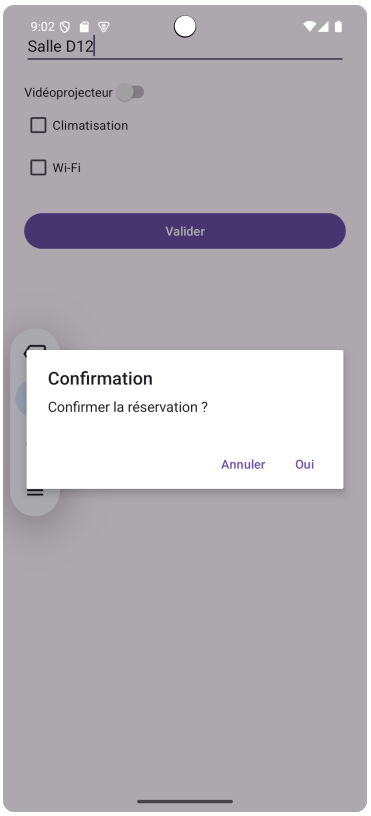

FIGURE 5 – Exemple d'interface de Confirmation

9 Événements utilisés (récapitulatif)

Composant	**Listener utilisé**
Button	OnClickListener
Spinner	OnItemSelectedListener
EditText	TextWatcher
Switch / CheckBox	OnCheckedChangeListener
Popup	DialogInterface.OnClickListener

10 Travail demandé

1. Créer le projet Android
2. Implémenter les 4 interfaces
3. Gérer correctement tous les événements
4. Ajouter le popup de confirmation
5. Tester tous les scénarios possibles

11 Questions (obligatoires)

1. Citer tous les listeners utilisés dans ce TP
2. Quel est le rôle du popup ?
3. Différence entre Toast et AlertDialog
4. Pourquoi ce TP est dit *événementiel* ?

Livrable

- Projet Android fonctionnel
- PDF contenant :
  - captures des écrans
  - réponses aux questions